

7

Logic, Shift, and Rotate Instructions

Overview

In this chapter we discuss instructions that can be used to change the bit pattern in a byte or word. The ability to manipulate bits is generally absent in high-level languages (except C), and is an important reason for programming in assembly language.

In section 7.1, we introduce the logic instructions AND, OR, XOR, and NOT. They can be used to clear, set, and examine bits in a register or variable. We use these instructions to do some familiar tasks, such as converting a lowercase letter to upper case, and some new tasks, such as determining if a register contains an even or odd number.

Section 7.2 covers the shift instructions. Bits can be shifted left or right in a register or memory location; when a bit is shifted out, it goes into CF. Because a left shift doubles a number and a right shift halves it, these instructions give us a way to multiply and divide by powers of 2. In Chapter 9, we'll use the MUL and DIV instructions for doing more general multiplication and division; however, these latter instructions are much slower than the shift instructions.

In section 7.3, the rotate instructions are covered. They work like the shifts, except that when a bit is shifted out one end of an operand it is put back in the other end. These instructions can be used in situations where we want to examine and/or change bits or groups of bits.

In section 7.4, we use the logic, shift, and rotate instructions to do binary and hexadecimal I/O. The ability to read and write numbers lets us solve a great variety of problems.

7.1

Logic Instructions

As noted earlier, the ability to manipulate individual bits is one of the advantages of assembly language. We can change individual bits in the computer by using logic operations. The binary values of 0 and 1 are treated as *false* and *true*, respectively. Figure 7.1 shows the truth tables for the logic operators AND, OR, XOR (exclusive OR), and NOT.

When a logic operation is applied to 8- or 16-bit operands, the result is obtained by applying the logic operation at each bit position.

Example 7.1 Perform the following logic operations:

1. 10101010 AND 11110000
2. 10101010 OR 11110000
3. 10101010 XOR 11110000
4. NOT 10101010

Solutions:

$$\begin{array}{r} 1. \quad 10101010 \\ \text{AND } 11110000 \\ \hline = 10100000 \end{array}$$

$$\begin{array}{r} 2. \quad 10101010 \\ \text{OR } 11110000 \\ \hline = 11111010 \end{array}$$

$$\begin{array}{r} 3. \quad 10101010 \\ \text{XOR } 11110000 \\ \hline = 01011010 \end{array}$$

$$\begin{array}{r} 4. \text{ NOT } 10101010 \\ = 01010101 \end{array}$$

Figure 7.1 Truth Tables for AND, OR, XOR, and NOT (0 = false, 1 = true)

a	b	a AND b	a OR b	a XOR b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

a	NOT a
0	1
1	0

7.1.1

AND, OR, and XOR Instructions

The **AND**, **OR**, and **XOR** instructions perform the named logic operations. The formats are

```
AND    destination,source
OR     destination,source
XOR    destination,source
```

The result of the operation is stored in the destination, which must be a register or memory location. The source may be a constant, register, or memory location. However, memory-to-memory operations are not allowed.

Effect on flags:

SF, ZF, PF reflect the result

AF is undefined

CF, OF = 0

One use of AND, OR, and XOR is to selectively modify the bits in the destination. To do this, we construct a source bit pattern known as a **mask**. The mask bits are chosen so that the corresponding destination bits are modified in the desired manner when the instruction is executed.

To choose the mask bits, we make use of the following properties of AND, OR, and XOR. From Figure 7.1, if b represents a bit (0 or 1)

```
b AND 1 = b    b OR 0 = b    b XOR 0 = b
b AND 0 = 0    b OR 1 = 1    b XOR 1 = ~b (complement of b)
```

From these, we may conclude that

1. The AND instruction can be used to **clear** specific destination bits while preserving the others. A 0 mask bit clears the corresponding destination bit; a 1 mask bit preserves the corresponding destination bit.
2. The OR instruction can be used to **set** specific destination bits while preserving the others. A 1 mask bit sets the corresponding destination bit; a 0 mask bit preserves the corresponding destination bit.
3. The XOR instruction can be used to **complement** specific destination bits while preserving the others. A 1 mask bit complements the corresponding destination bit; a 0 mask bit preserves the corresponding destination bit.

Example 7.2 Clear the sign bit of AL while leaving the other bits unchanged.

Solution: Use the AND instruction with $01111111b = 7Fh$ as the mask. Thus,

```
AND    AL, 7Fh
```

Example 7.3 Set the most significant and least significant bits of AL while preserving the other bits.

Solution: Use the OR instruction with 10000001b = 81h as the mask. Thus,

```
OR    AL, 81h
```

Example 7.4 Change the sign bit of DX.

Solution: Use the XOR instruction with a mask of 8000h. Thus,

```
XOR    DX, 8000h
```

Note: to avoid typing errors, it's best to express the mask in hex rather than binary, especially if the mask would be 16 bits long.

The logic instructions are especially useful in the following frequently occurring tasks.

Converting an ASCII Digit to a Number

We've seen that when a program reads a character from the keyboard, AL gets the ASCII code of the character. This is also true of digit characters. For example, if the "5" key is pressed, AL gets 35h instead of 5. To get 5 in AL, we could do this:

```
SUB    AL, 30h
```

Another method is to use the AND instruction to clear the high nibble (high four bits) of AL:

```
AND    AL, 0Fh
```

Because the codes of "0" to "9" are 30h to 39h, this method will convert any ASCII digit to a decimal value.

By using the logic instruction AND instead of SUB, we emphasize that we're modifying the bit pattern of AL. This is helpful in making the program more readable.

The reverse problem of converting a stored decimal digit to its ASCII code is left as an exercise.

Converting a Lowercase Letter to Upper Case

The ASCII codes of "a" to "z" range from 61h to 7Ah; the codes of "A" to "Z" go from 41h to 5Ah. Thus for example, if DL contains the code of a lowercase letter, we could convert to upper case by executing

```
SUB    DL, 20h
```

This method was used in Chapter 4. However, if we compare the binary codes of corresponding lowercase and uppercase letters

<i>Character</i>	<i>Code</i>	<i>Character</i>	<i>Code</i>
a	01100001	A	01000001
b	01100010	B	01000010
.	.	.	.
.	.	.	.
.	.	.	.
z	01111010	Z	01011010

it is apparent that to convert lower to upper case we need only clear bit 5. This can be done by using an AND instruction with the mask 11011111b, or 0DFh. So if the lowercase character to be converted is in DL, execute

```
AND    DL, 0DFh
```

The reverse problem of conversion from upper to lower case is left as an exercise.

Clearing a Register

We already know two ways to clear a register. For example, to clear AX we could execute

```
MOV    AX, 0
```

or

```
SUB    AX, AX
```

Using the fact that $1 \text{ XOR } 1 = 0$ and $0 \text{ XOR } 0 = 0$, a third way is

```
XOR    AX, AX
```

The machine code of the first method is three bytes, versus two bytes for the latter two methods, so the latter are more efficient. However, because of the prohibition on memory-to-memory operations, the first method must be used to clear a memory location.

Testing a Register for Zero

Because $1 \text{ OR } 1 = 1$, $0 \text{ OR } 0 = 0$, it might seem like a waste of time to execute an instruction like

```
OR     CX, CX
```

because it leaves the contents of CX unchanged. However, it affects ZF and SF, and in particular if CX contains 0 then $ZF = 1$. So it can be used as an alternative to

```
CMP    CX, 0
```

to test the contents of a register for zero, or to check the sign of the contents.

7.1.2

NOT Instruction

The **NOT** instruction performs the one's complement operation on the destination. The format is

```
NOT    destination
```

There is no effect on the status flags.

Example 7.5 Complement the bits in AX.

Solution:

```
NOT    AX
```


7.1.3

TEST Instruction

The **TEST** instruction performs an AND operation of the destination with the source but does not change the destination contents. The purpose of the TEST instruction is to set the status flags. The format is

TEST destination,source

Effect on flags

SF, ZF, PF reflect the result

AF is undefined

CF, OF = 0

Examining Bits

The TEST instruction can be used to examine individual bits in an operand. The mask should contain 1's in the bit positions to be tested and 0's elsewhere. Because $1 \text{ AND } b = b$, $0 \text{ AND } b = 0$, the result of

TEST destination,mask

will have 1's in the tested bit positions if and only if the destination has 1's in these positions; it will have 0's elsewhere. If destination has 0's in all the tested positions, the result will be 0 and so ZF = 1.

Example 7.6 Jump to label BELOW if AL contains an even number.

Solution: Even numbers have a 0 in bit 0. Thus, the mask is 00000001b = 1.

```
TEST AL,1           ;is AL even?
JZ  BELOW          ;yes, go to BELOW
```

7.2

Shift Instructions

The shift and rotate instructions shift the bits in the destination operand by one or more positions either to the left or right. For a shift instruction, the bits shifted out are lost; for a rotate instruction, bits shifted out from one end of the operand are put back into the other end. The instructions have two possible formats. For a single shift or rotate, the form is

Opcode destination,1

For a shift or rotate of N positions, the form is

Opcode destination,CL

where CL contains N . In both cases, destination is an 8- or 16-bit register or memory location. Note that for Intel's more advanced processors, a shift or rotate instruction also allows the use of an 8-bit constant.

As we'll see presently, these instructions can be used to multiply and divide by powers of 2, and we will use them in programs for binary and hex I/O.

7.2.1

Left Shift Instructions**The SHL Instruction**

The **SHL** (shift left) instruction shifts the bits in the destination to the left. The format for a single shift is

```
SHL destination, 1
```

A 0 is shifted into the rightmost bit position and the msb is shifted into CF (Figure 7.2). If the shift count N is different from 1, the instruction takes the form

```
SHL destination, CL
```

where CL contains N . In this case, N single left shifts are made. The value of CL remains the same after the shift operation.

Effect on flags

SF, PF, ZF reflect the result

AF is undefined

CF = last bit shifted out

OF = 1 if result changes sign on last shift

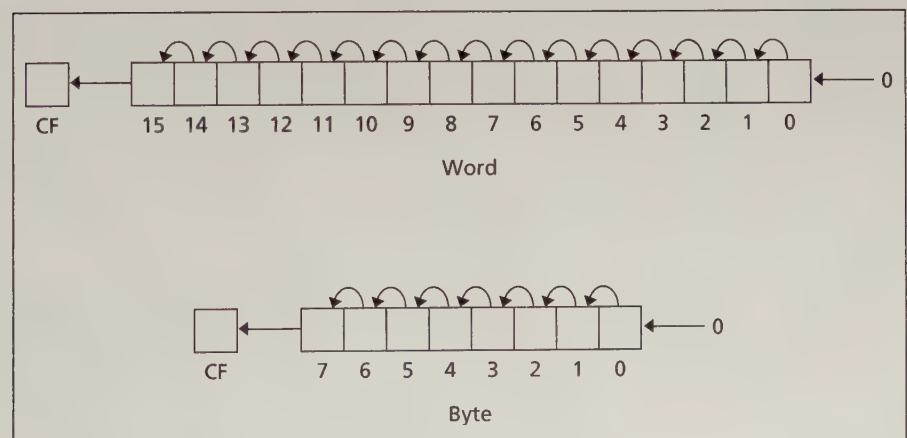
Example 7.7 Suppose DH contains 8Ah and CL contains 3. What are the values of DH and of CF after the instruction SHL DH, CL is executed?

Solution: The binary value of DH is 10001010. After 3 left shifts, CF will contain 0. The new contents of DH may be obtained by erasing the leftmost three bits and adding three zero bits to the right end, thus 01010000b = 50h.

Multiplication by Left Shift

Consider the decimal number 235. If each digit is shifted left one position and a 0 attached to the right end, we get 2350; this is the same as multiplying 235 by ten.

Figure 7.2 SHL and SAL



In the same way, a left shift on a binary number multiplies it by 2. For example, suppose that AL contains $5 = 00000101b$. A left shift gives $00001010b = 10d$, thus doubling its value. Another left shift yields $00010100 = 20d$, so it is doubled again.

The SAL instruction

Thus, the SHL instruction can be used to multiply an operand by multiples of 2. However, to emphasize the arithmetic nature of the operation, the opcode **SAL** (shift arithmetic left) is often used in instances where numeric multiplication is intended. Both instructions generate the same machine code.

Negative numbers can also be multiplied by powers of 2 by left shifts. For example, if AX is FFFFh (-1), then shifting three times will yield AX = FFF8h (-8).

Overflow

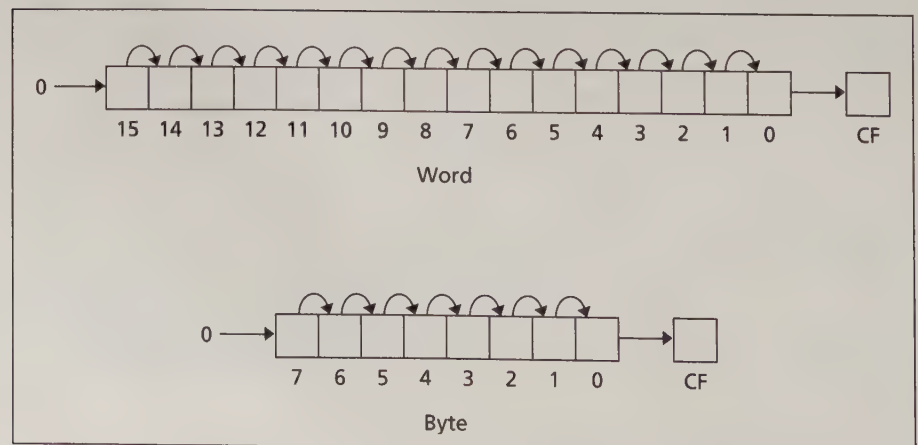
When we treat left shifts as multiplication, overflow may occur. For a single left shift, CF and OF accurately indicate unsigned and signed overflow, respectively. However, the overflow flags are not reliable indicators for a multiple left shift. This is because a multiple shift is really a series of single shifts, and CF, OF only reflect the result of the last shift. For example, if BL contains 80h, CL contains 2 and we execute SHL BL,CL, then CF = OF = 0 even though both signed and unsigned overflow occur.

Example 7.8 Write some code to multiply the value of AX by 8. Assume that overflow will not occur.

Solution: To multiply by 8, we need to do three left shifts.

```
MOV CL,3           ;number of shifts to do
SAL AX,CL          ;multiply by 8
```

Figure 7.3 SHR



7.2.2

Right Shift Instructions

The SHR Instruction

The instruction **SHR** (shift right) performs right shifts on the destination operand. The format for a single shift is

```
SHR destination, 1
```

A 0 is shifted into the msb position, and the rightmost bit is shifted into CF. See Figure 7.3. If the shift count N is different from 1, the instruction takes the form

```
SHR destination, CL
```

where CL contains N . In this case N single right shifts are made.

The effect on the flags is the same as for SHL.

Example 7.9 Suppose DH contains 8Ah and CL contains 2. What are the values of DH and CF after the instruction SHR DH, CL is executed?

Solution: The value of DH in binary is 10001010. After two right shifts, CF = 1. The new value of DH is obtained by erasing the rightmost two bits and adding two 0 bits to the left end, thus DH = 00100010b = 22h.

The SAR Instruction

The **SAR** instruction (shift arithmetic right) operates like SHR, with one difference: the msb retains its original value. See Figure 7.4. The syntax is

```
SAR destination, 1
```

and

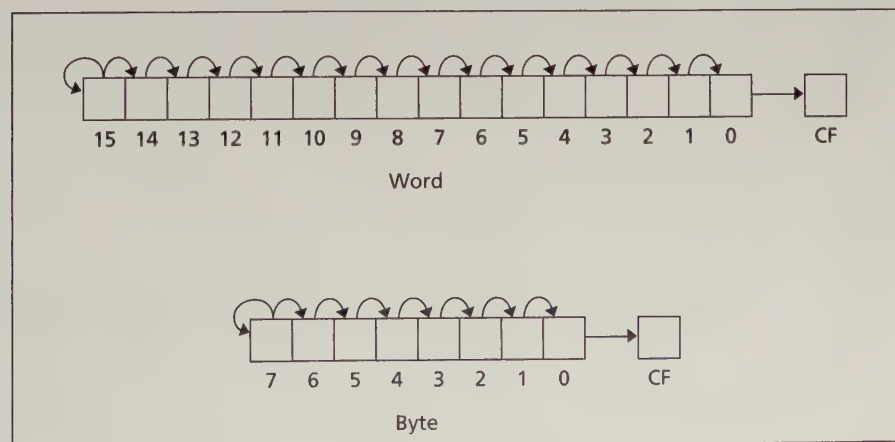
```
SAR destination, CL
```

The effect on flags is the same as for SHR.

Division by Right Shift

Because a left shift doubles the destination's value, it's reasonable to guess that a right shift might divide it by 2. This is correct for even numbers.

Figure 7.4 SAR



For odd numbers, a right shift halves it and rounds down to the nearest integer. For example, if BL contains 00000101b = 5, then after a right shift BL will contain 00000010 = 2.

Signed and Unsigned Division

In doing division by right shifts, we need to make a distinction between signed and unsigned numbers. If an unsigned interpretation is being given, SHR should be used. For a signed interpretation, SAR must be used, because it preserves the sign.

Example 7.10 Use right shifts to divide the unsigned number 65143 by 4. Put the quotient in AX.

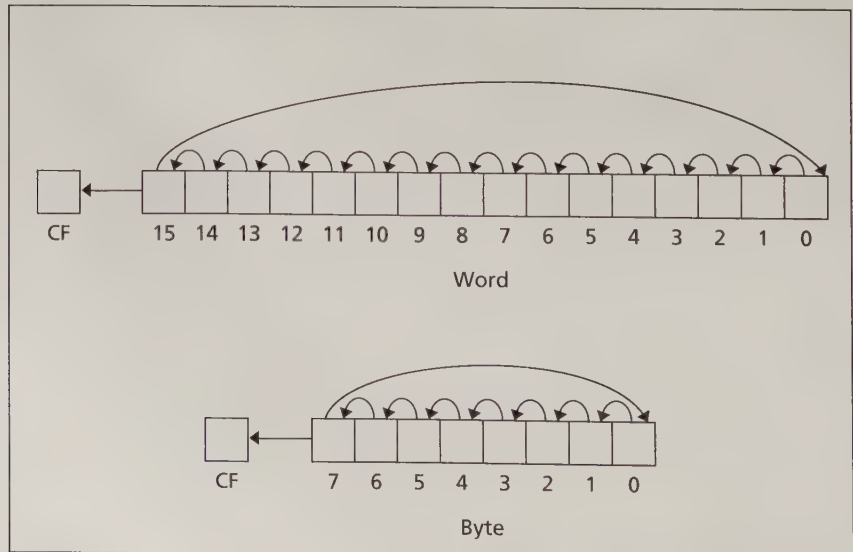
Solution: To divide by 4, two right shifts are needed. Since the dividend is unsigned, we use SHR. The code is

```
MOV  AX,65143      ;AX has number
MOV  CL,2          ;CL has number of right shifts
SHR  AX,CL         ;divide by 4
```

Example 7.11 If AL contains -15, give the decimal value of AL after SAR AL,1 is performed.

Solution: Execution of SAR AL,1 divides the number by 2 and rounds down. Dividing -15 by 2 yields -7.5, and after rounding down we get -8. In terms of the binary contents, we have -15 = 11110001b. After shifting, we have 11111000b = -8.

Figure 7.5 ROL



More General Multiplication and Division

We've seen that multiplication and division by powers of 2 can be accomplished by left and right shifts. Multiplication by other numbers, such as 10d, can be done by a combination of shifting and addition (see Chapter 8).

In Chapter 9, we cover the MUL and IMUL, DIV and IDIV instructions. They are not limited to multiplication and division by powers of 2, but are much slower than the shift instructions.

7.3 Rotate Instructions

Rotate Left

The instruction **ROL** (rotate left) shifts bits to the left. The msb is shifted into the rightmost bit. The CF also gets the bit shifted out of the msb. You can think of the destination bits forming a circle, with the least significant bit following the msb in the circle. See Figure 7.5. The syntax is

```
ROL    destination,1
```

and

```
ROL    destination,CL
```

Rotate Right

The instruction **ROR** (rotate right) works just like ROL, except that the bits are rotated to the right. The rightmost bit is shifted into the msb, and also into the CF. See Figure 7.6. The syntax is

```
ROR    destination,1
```

and

```
ROR    destination,CL
```

Figure 7.6 ROR

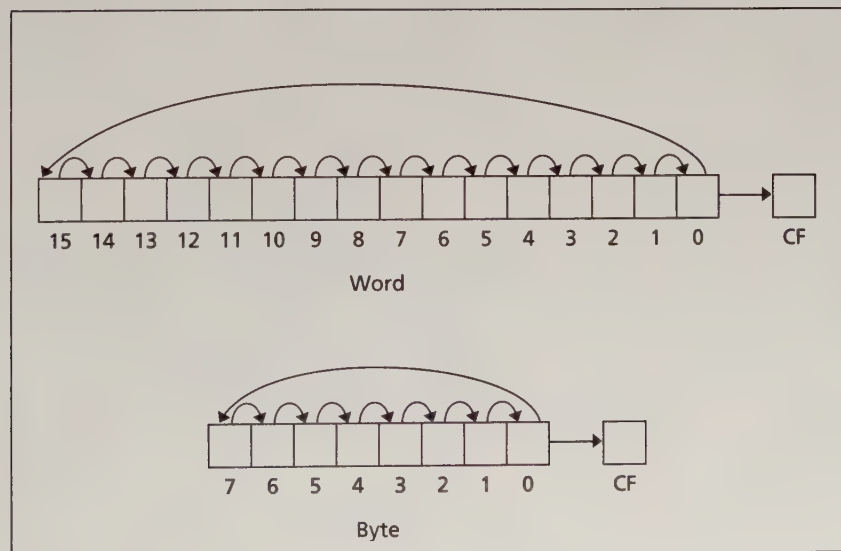
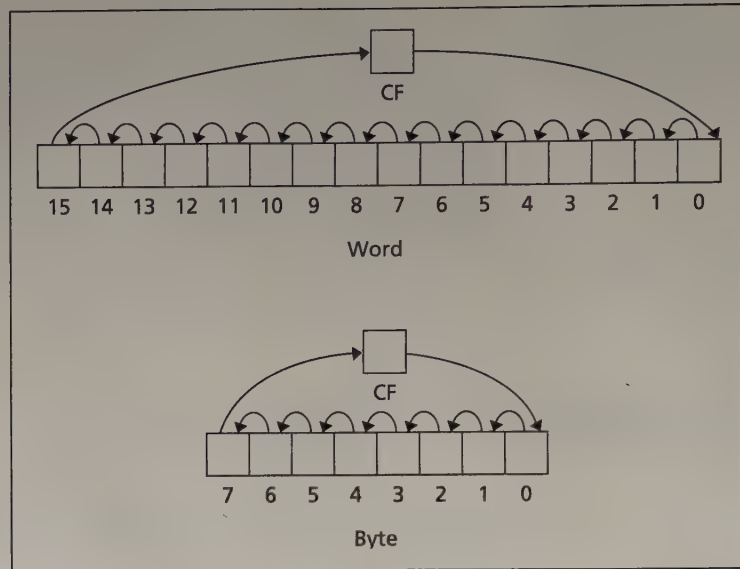


Figure 7.7 RCL



In ROL and ROR, CF reflects the bit that is rotated out. The next example shows how this can be used to inspect the bits in a byte or word, without changing the contents.

Example 7.12 Use ROL to count the number of 1 bits in BX, without changing BX. Put the answer in AX.

Solution:

```

                                XOR  AX,AX      ;AX counts bits
                                MOV  CX,16      ;loop counter
TOP:
                                ROL  BX,1       ;C F = bit rotated out
                                JNC  NEXT       ;0 bit
                                INC  AX        ;1 bit, increment total
NEXT:
                                LOOP TOP        ;loop until done

```

In this example, we used JNC (Jump if No Carry), which causes a jump if CF = 0. In section 7.4, we use ROL to output the contents of a register in binary.

Rotate Carry Left

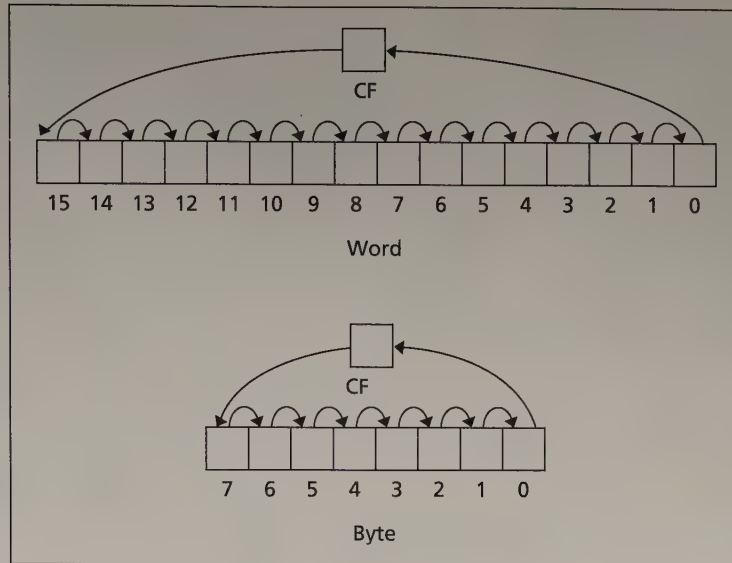
The instruction **RCL** (Rotate through Carry Left) shifts the bits of the destination to the left. The msb is shifted into the CF, and the previous value of CF is shifted into the rightmost bit. In other words, RCL works like just like ROL, except that CF is part of the circle of bits being rotated. See Figure 7.7. The syntax is

```

RCL  destination,1
and
RCL  destination,CL

```

Figure 7.8 RCR



Rotate Carry Right

The instruction **RCR** (Rotate through Carry Right) works just like RCL, except that the bits are rotated to the right. See Figure 7.8. The syntax is

```
RCR destination,1
```

and

```
RCR destination,CL
```

Example 7.13 Suppose DH contains 8Ah, CF = 1, and CL contains 3. What are the values of DH and CF after the instruction RCR DH,CL is executed?

Solution:

	<i>CF</i>	<i>DH</i>
initial values	1	10001010
after 1 right rotation	0	11000101
after 2 right rotations	1	01100010
after 3 right rotations	0	10110001 = B1h

Effect of the rotate instructions on the flags

SF, PF, ZF reflect the result

AF is undefined

CF = last bit shifted out

OF = 1 if result changes sign on the last rotation

An Application: Reversing a Bit Pattern

As an application of the shift and rotate instructions, let's consider the problem of reversing the bit pattern in a byte or word. For example, if AL contains 11011100, we want to make it 00111011.

An easy way to do this is to use SHL to shift the bits out the left end of AL into CF, and then use RCR to move them into the left end of another register; for example, BL. If this is done eight times, BL will contain the reversed bit pattern and it can be copied back into AL. The code is

```

MOV     CX,8      ;number of operations to do
REVERSE:
        SHL     AL,1      ;get a bit into CF
        RCR     BL,1      ;rotate it into BL
        LOOP    REVERSE   ;loop until done
        MOV     AL,BL     ;AL gets reversed pattern

```

7.4

Binary and Hex I/O

One useful application of the shift and rotate instructions is in binary and hex I/O.

Binary Input

For binary input, we assume a program reads in a binary number from the keyboard, followed by a carriage return. The number actually is a character string of 0's and 1's. As each character is entered, we need to convert it to a bit value, and collect the bits in a register. The following algorithm reads a binary number from the keyboard and stores its value in BX.

Algorithm for Binary Input

```

Clear BX      /* BX will hold binary value */
Input a character /* '0' or '1' */
WHILE character <> CR DO
    Convert character to binary value
    Left shift BX
    Insert value into lsb of BX
    Input a character
END_WHILE

```

Demonstration (for input 110)

```

Clear BX
BX = 0000 0000 0000 0000
Input character '1', convert to 1
Left shift BX
BX = 0000 0000 0000 0000
Insert value into lsb
BX = 0000 0000 0000 0001
Input character '1', convert to 1
Left shift BX
BX = 0000 0000 0000 0010
Insert value into lsb
BX = 0000 0000 0000 0011

```

```

Input character '0', convert to 0
Left shift BX
    BX = 0000 0000 0000 0110
Insert value into lsb
    BX = 0000 0000 0000 0110
BX contains 110b.

```

The algorithm assumes (1) input characters are either "0", "1", or CR, and (2) at most 16 binary digits are input. As a new digit is input, the previous bits in BX must be shifted to the left to make room; then an OR operation can be used to insert the new bit into BX. The assembly instructions are

```

                                XOR  BX,BX      ;clear BX
                                MOV  AH,1       ;input char function
                                INT  21H        ;read a character
WHILE_:
                                CMP  AL,0DH      ;CR?
                                JE    END_WHILE ;yes, done
                                AND  AL,0FH      ;no, convert to binary value
                                SHL  BX,1       ;make room for new value
                                OR   BL,AL      ;put value into BX
                                INT  21H        ;read a character
                                JMP  WHILE_      ;loop back
END_WHILE:

```

Binary Output

Outputting the contents of BX in binary also involves the shift operation. Here we only give an algorithm; the assembly code is left to be done as an exercise.

Algorithm for Binary Output

```

FOR 16 times DO
    Rotate left BX /* BX holds output value,
                    put msb into CF */
    IF CF = 1
    THEN
        output '1'
    ELSE
        output '0'
    END_IF
END_FOR

```

Hex Input

Hex input consists of digits ("0" to "9") and letters ("A" to "F") followed by a carriage return. For simplicity, we assume that (1) only upper-case letters are used, and (2) the user inputs no more than four hex characters. The process of converting characters to binary values is more involved than it was for binary input, and BX must be shifted four times to make room for a hex value.

Algorithm for Hex Input

```

Clear BX      /* BX will hold input value */
input hex character
WHILE character <> CR DO
    convert character to binary value
    left shift BX 4 times
    insert value into lower 4 bits of BX
    input a character
END_WHILE

```

Demonstration (for input 6AB)

```

Clear BX
BX = 0000 0000 0000 0000
Input '6', convert to 0110
Left shift BX 4 times
BX = 0000 0000 0000 0000
Insert value into lower 4 bits of BX
BX = 0000 0000 0000 0110
Input 'A', convert to Ah = 1010
Left shift BX 4 times
BX = 0000 0000 0110 0000
Insert value into lower 4 bits of BX
BX = 0000 0000 0110 1010
Input 'B', convert to 1011
Left shift BX 4 times
BX = 0000 0110 1010 0000
Insert value into lower 4 bits of BX
BX = 0000 0110 1010 1011
BX contains 06ABh.

```

Here is the code:

```

                                XOR  BX,BX      ;clear BX
                                MOV  CL,4       ;counter for 4 shifts
                                MOV  AH,1       ;input character function
                                INT   21H       ;input a character

WHILE_:
                                CMP  AL,0DH     ;CR?
                                JE    END_WHILE ;yes, exit
                                ;convert character to binary value
                                CMP  AL,39H     ;a digit?
                                JG    LETTER    ;no, a letter
                                ;input is a digit
                                AND  AL,0FH     ;convert digit to binary value
                                JMP  SHIFT      ;go to insert in BX

LETTER:
                                SUB  AL,37H     ;convert letter to binary value

SHIFT:
                                SHL  BX,CL      ;make room for new value
                                ;insert value into BX
                                OR   BL,AL     ;put value into low 4 bits
                                                ;of BX
                                INT   21H      ;input a character
                                JMP  WHILE_    ;loop until CR

END_WHILE:

```

Note that the program does not check for valid input characters.

Hex Output

BX contains 16 bits, which equal four hex digit values. To output the contents of BX, we start from the left and get hold of each digit, convert it to a hex character, and output it. The algorithm which follows is similar to that for binary output.

Algorithm for Hex Output

```
FOR 4 times DO
  Move BH to DL      /* BX holds output value */
  shift DL 4 times to the right
  IF DL < 10
    THEN
      convert to character in '0'..'9'
    ELSE
      convert to character in 'A'..'F'
  END_IF
  output character
  Rotate BX left 4 times
END_FOR
```

Demonstration (BX Contains 4CA9h)

```
BX = 4CA9h = 0100 1100 1010 1001
Move BH to DL
DL = 0100 1100
Shift DL 4 times to the right
DL = 0000 0100
Convert to character and output
DL = 0011 0100 = 34h = '4'
Rotate BX left 4 times
BX = 1100 1010 1001 0100
Move BH to DL
DL = 1100 1010
Shift DL 4 times to the right
DL = 0000 1100
Convert to character and output
DL = 0100 0011 = 43h = 'C'
Rotate BX left 4 times
BX = 1010 1001 0100 1100
Move BH to DL
DL = 1010 1001
Shift DL 4 times to the right
DL = 0000 1010
Convert to character and output
DL = 0100 0010 = 42h = 'B'
Rotate BX left 4 times
BX = 1001 0100 1100 1010
Move BH to DL
DL = 1001 0100
Shift DL 4 times to the right
```

```

DL = 0000 1001
Convert to character and output
DL = 0011 1001 = 39h = '9'
Rotate BX 4 times to the left
BX = 0100 1100 1010 1001 = original contents

```

Coding the algorithm is left to be done as an exercise.

Summary

- The five logic instructions are AND, OR, NOT, XOR, and TEST.
- The AND instruction can be used to clear individual bits in the destination.
- The OR instruction is useful in setting individual bits in the destination. It can also be used to test the destination for zero.
- The XOR instruction can be used to complement individual bits in the destination. It can also be used to zero out the destination.
- The NOT instruction performs the one's complement operation on the destination.
- The TEST instruction can be used to examine individual bits of the destination. For example, it can determine if the destination contains an even or odd number.
- SAL and SHL shift each destination bit left one place. The most significant bit goes into CF, and a 0 is shifted into the least significant bit.
- SHR shifts each destination bit right one place. The least significant bit goes into CF, and a 0 is shifted into the most significant bit.
- SAR operates like SHR, except that the value of the most significant bit is preserved.
- The shift instructions can be used to do multiplication and division by 2. SHL and SAL double the destination's value unless overflow occurs. SHR and SAR halve the destination's value if it is even; if odd, they halve the destination's value and round down to the nearest integer. SHR should be used for unsigned arithmetic, and SAR for signed arithmetic.
- ROL shifts each destination bit left one position; the most significant bit is rotated into the least significant bit. For ROR, each bit goes right one position, and the least significant bit rotates into the most significant bit. For both instructions, CF gets the last bit rotated out.
- RCL and RCR operate like ROL and ROR, except that a bit rotated out goes into CF, and the value of CF rotates into the destination.
- Multiple shifts and rotates can be performed. CL must contain the number of times the shift or rotate is to be executed.
- The shift and rotate instructions are useful in doing binary and hex I/O.

Glossary

clear	Change a value to 0
complement	Change from a 0 to a 1 or from a 1 to a 0
mask	A bit pattern used in logical operations to clear, set, or test specific bits in an operand
set	Change a bit value to a 1

New Instructions

AND	RCR	SAR
NOT	ROL	SHR
OR	ROR	TEST
RCL	SAL/SHL	XOR

Exercises

- Perform the following logic operations
 - 10101111 AND 10001011
 - 10110001 OR 01001001
 - 01111100 XOR 11011010
 - NOT 01011110
- Give a logic instruction to do each of the following.
 - Clear the even-numbered bits of AX, leaving the other bits unchanged.
 - Set the most and least significant bits of BL, leaving the other bits unchanged.
 - Complement the msb of DX, leaving the other bits unchanged.
 - Replace the value of the word variable WORD1 by its one's complement.
- Use the TEST instruction to do each of the following.
 - Set ZF if the contents of AX is zero.
 - Clear ZF if BX contains an odd number.
 - Set SF if DX contains a negative number.
 - Set ZF if DX contains a zero or positive number.
 - Set PF if BL contains an even number of 1 bits.
- Suppose AL contains 11001011b and CF = 1. Give the new contents of AL after each of the following instructions is executed. Assume the preceding initial conditions for each part of this question.
 - SHL AL,1
 - SHR AL,1
 - ROL AL,CL if CL contains 2
 - ROR AL,CL if CL contains 3
 - SAR AL,CL if CL contains 2
 - RCL AL,1
 - RCR AL,CL if CL contains 3

5. Write one or more instructions to do each of the following. Assume overflow does not occur.
 - a. Double the value of byte variable B5.
 - b. Multiply the value of AL by 8.
 - c. Divide 32142 by 4 and put the quotient in AX.
 - d. Divide -2145 by 16 and put the quotient in BX.
6. Write instructions to do each of the following:
 - a. Assuming AL has a value less than 10, convert it to a decimal character.
 - b. Assuming DL contains the ASCII code of an uppercase letter, convert it to lower case.
7. Write instructions to do each of the following.
 - a. Multiply the value of BL by 10d. Assume overflow does not occur.
 - b. Suppose AL contains a positive number. Divide AL by 8, and put the remainder in AH. (*Hint: use ROR.*)

Programming Exercises

8. Write a program that prompts the user to enter a character, and on subsequent lines prints its ASCII code in binary, and the number of 1 bits in its ASCII code.

Sample execution:

```
TYPE A CHARACTER: A
THE ASCII CODE OF A IN BINARY IS 01000001
THE NUMBER OF 1 BITS IS 2
```

9. Write a program that prompts the user to enter a character and prints the ASCII code of the character in hex on the next line. Repeat this process until the user types a carriage return.

Sample execution:

```
TYPE A CHARACTER: Z
THE ASCII CODE OF Z IN HEX IS 5A
TYPE A CHARACTER:
```

10. Write a program that prompts the user to type a hex number of four hex digits or less, and outputs it in binary on the next line. If the user enters an illegal character, he or she should be prompted to begin again. Accept only uppercase letters.

Sample execution:

```
TYPE A HEX NUMBER (0 TO FFFF): 1a
ILLEGAL HEX DIGIT, TRY AGAIN: 1ABC
IN BINARY IT IS 0001101010111100
```

Your program may ignore any input beyond four characters.

11. Write a program that prompts the user to type a binary number of 16 digits or less, and outputs it in hex on the next line. If the user enters an illegal character, he or she should be prompted to begin again.

Sample execution:

```
TYPE A BINARY NUMBER, UP TO 16 DIGITS: 11100001
IN HEX IT IS E1
```

Your program may ignore any input beyond 16 characters.

12. Write a program that prompts the user to enter two binary numbers of up to 8 digits each, and prints their sum on the next line in binary. If the user enters an illegal character, he or she should be prompted to begin again. Each input ends with a carriage return.

Sample execution:

```
TYPE A BINARY NUMBER, UP TO 8 DIGITS: 11001010
TYPE A BINARY NUMBER, UP TO 8 DIGITS: 10011100
THE BINARY SUM IS 101100110
```

13. Write a program that prompts the user to enter two unsigned hex numbers, 0 to FFFFh, and prints their sum in hex on the next line. If the user enters an illegal character, he or she should be prompted to begin again. Your program should be able to handle the possibility of unsigned overflow. Each input ends with a carriage return.

Sample execution:

```
TYPE A HEX NUMBER, 0 - FFFF: 21AB
TYPE A HEX NUMBER, 0 - FFFF: FE03
THE SUM IS 11FAE
```

14. Write a program that prompts the user to enter a string of decimal digits, ending with a carriage return, and prints their sum in hex on the next line. If the user enters an illegal character, he or she should be prompted to begin again.

Sample execution:

```
ENTER A DECIMAL DIGIT STRING: 1299843
THE SUM OF THE DIGITS IN HEX IS 0024
```

